Project review

Security Operations Center (SOC) – EDR Grid

Project Title: Enterprise EDR & Threat Hunting Grid
Product Brand Name: Sheth jainam

Week 1 Report

1. Objective:
The objective of Week 1 was to **design and deploy the foundational infrastructure** required for the Enterprise Endpoint Detection and Response (EDR) grid, named **Sentient Shield**. The purpose of this setup was to establish a centralized monitoring system capable of collecting, analyzing, and managing security logs from multiple endpoints across different operating systems.
The deployment utilized **Wazuh**, an open-source security monitoring and threat detection platform, to function as the central management system. Additionally, Sysmon was installed on the Windows host to provide enhanced system telemetry including process creation, network connections, and file activity.
The key objectives for this week included:

- Deploying the Wazuh Manager on Ubuntu 24.04
- Installing Wazuh agents on Kali Linux
- Installing Sysmon on Windows 11 for advanced logging
- Ensuring that all agents successfully connect to the manager
- Verifying that agents are sending heartbeat signals and logs

2. Infrastructure Architecture-
The Week 1 infrastructure was designed to **simulate a real-world enterprise monitoring** environment where multiple endpoints send security logs to a centralized SOC platform.
The environment consisted of three main systems:

System role	Operating system
Wazuh Manager- Central security monitoring server	Ubuntu 24.04
Linux Agent- Endpoint monitored by Wazuh	Kali linux
Windows Agent- Endpoint with enhanced telemetry	Windows 11

Since the systems were located on different networks, **Tailscale** was used to create a secure mesh VPN that allowed all devices to communicate as if they were on the same private network.

The Tailscale network ensured secure connectivity between:

- Ubuntu Wazuh Manager
- Kali Linux Agent
- Windows 11 Agent

Due to the free version of Tailscale allowing only three users, the fourth group member assisted with command execution, verification, and configuration support during deployment.

3. Wazuh Manager Deployment-

- The Wazuh Manager was deployed on an Ubuntu 24.04 system which served as the central monitoring server responsible for collecting and analyzing logs from connected agents.
- The installation was performed using the official Wazuh installation script.

Installation Commands:

→ curl -sO https://packages.wazuh.com/4.x/wazuh-install.sh
sudo bash wazuh-install.sh -a

The installation script automatically installed and configured the following components:

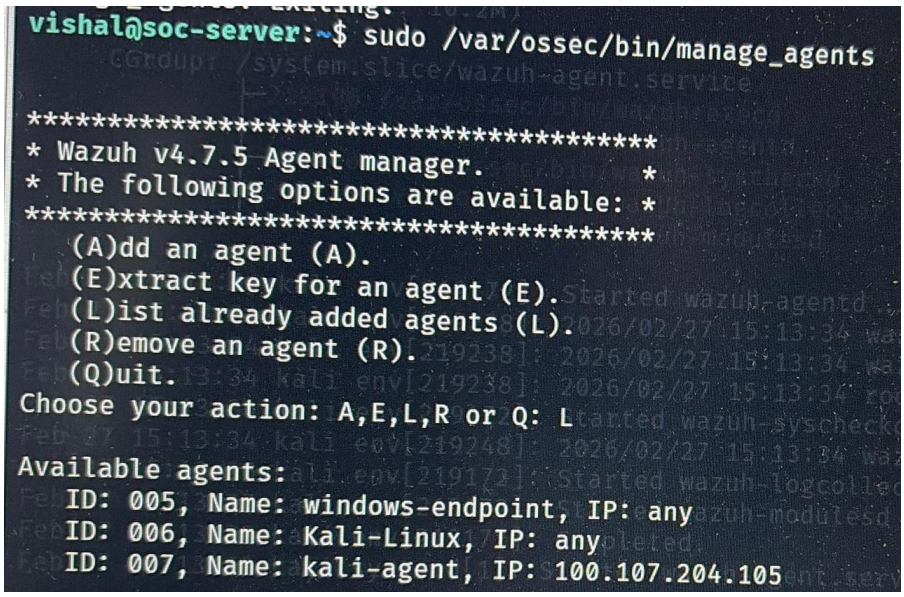
- **Wazuh Manager**
- **Wazuh Indexer**
- **Wazuh Dashboard**

The Wazuh Dashboard provides a graphical interface where administrators can monitor agents, security events, and alerts generated by the system.

4. Agent Deployment-

4.1 Linux Agent – Kali Linux:

- A Kali Linux system was configured as a Wazuh agent to send system logs and security data to the Wazuh Manager.
- The Wazuh Agent was installed and configured to communicate with the manager using the Tailscale IP address of the Ubuntu server.
- Once the agent was registered with the manager, it began transmitting system logs and heartbeat signals to confirm connectivity.

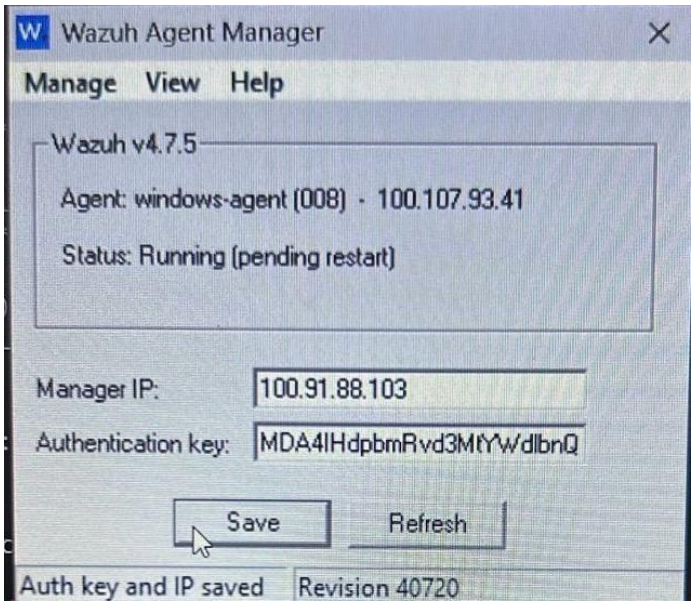


4.2 Windows Agent – Windows 11

A Windows 11 machine was deployed as the second monitored endpoint in the environment.
The following components were installed:

- **Wazuh Agent**
- **Sysmon**

The Wazuh Agent enabled communication between the Windows system and the Wazuh Manager.



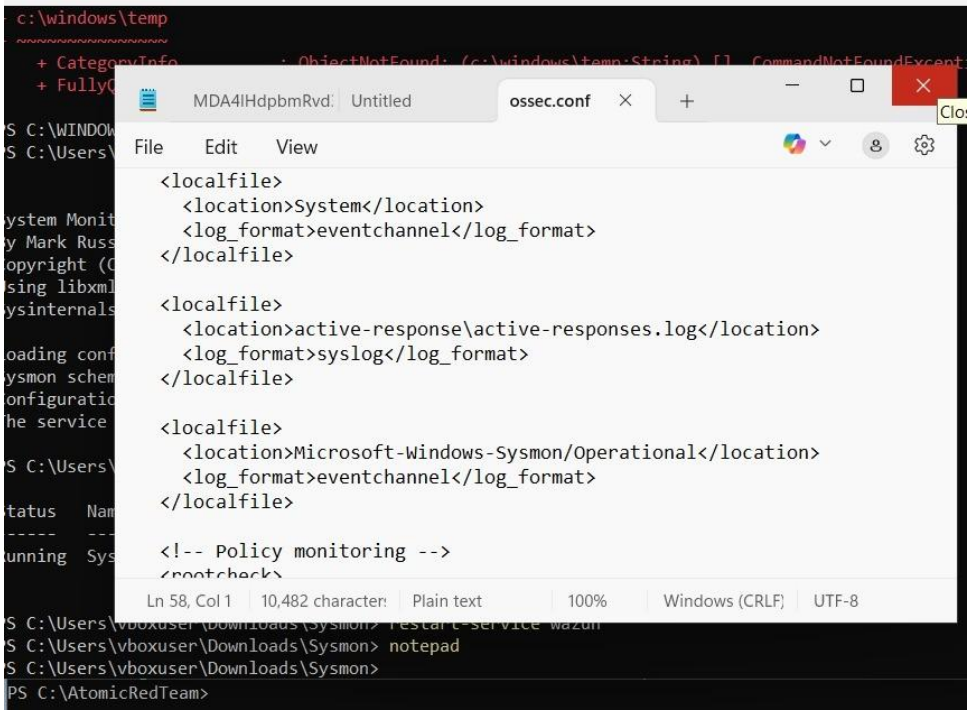
Sysmon was installed to generate detailed telemetry about system activity, including:

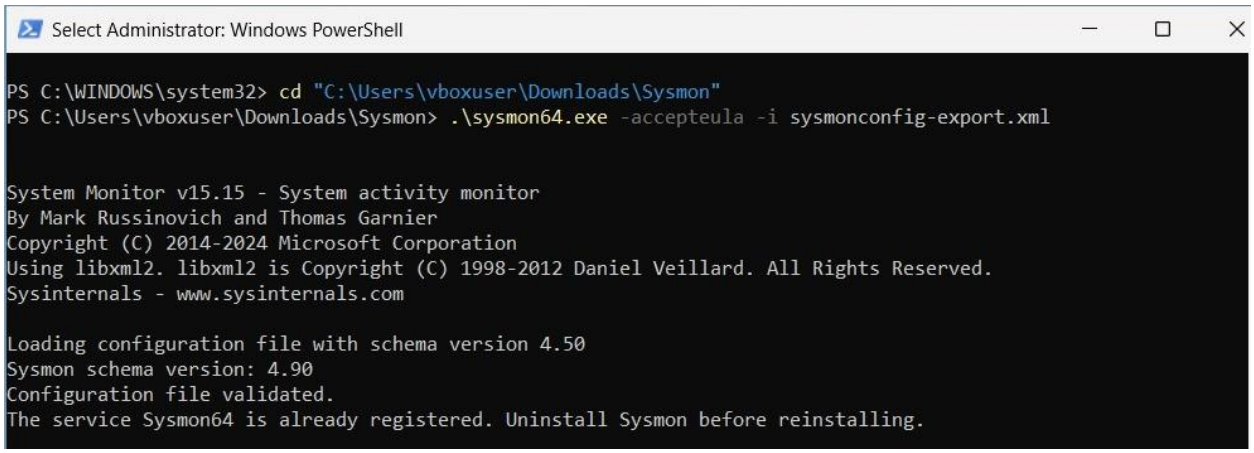
- Process creation events
- Network connection logs

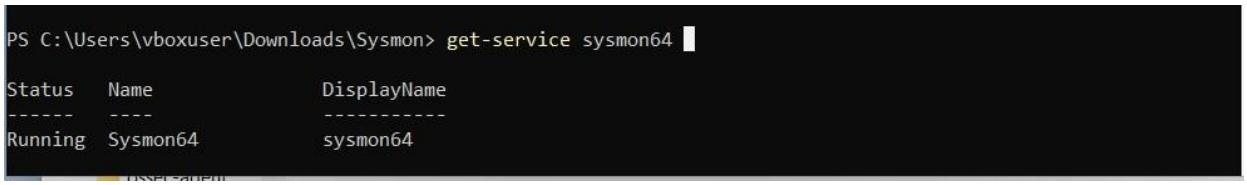
Project review

- File creation and modification activity
- System configuration changes

These logs were forwarded to the Wazuh Manager where they could be analyzed and monitored.







5. Network Connectivity Verification-

To ensure that all machines were connected within the same virtual network, the team used Tailscale.
The following command was used to verify connectivity between devices:
→ tailscale status
This command displayed all devices connected to the mesh VPN network along with their assigned Tailscale IP addresses.
Successful connectivity confirmed that all systems could communicate with the Wazuh Manager.

```
vishal@soc-server:~$ sudo tailscale status
100.91.88.103    soc-server  mvishal-02@       linux
100.107.204.105  kali        sanjanayoga6@     linux
100.107.93.41    windows     mohanmonisha132@  windows
vishal@soc-server:~$
```

6. Agent Status Verification-

- After installing and configuring the agents, their connectivity to the manager was verified.
- The following command was executed on the manager to display the list of connected agents:
  → sudo /var/ossec/bin/agent_control -l
- This command displayed the status of all registered agents and confirmed whether they were active.
- Both the Kali Linux agent and the Windows 11 agent were successfully registered with the manager.

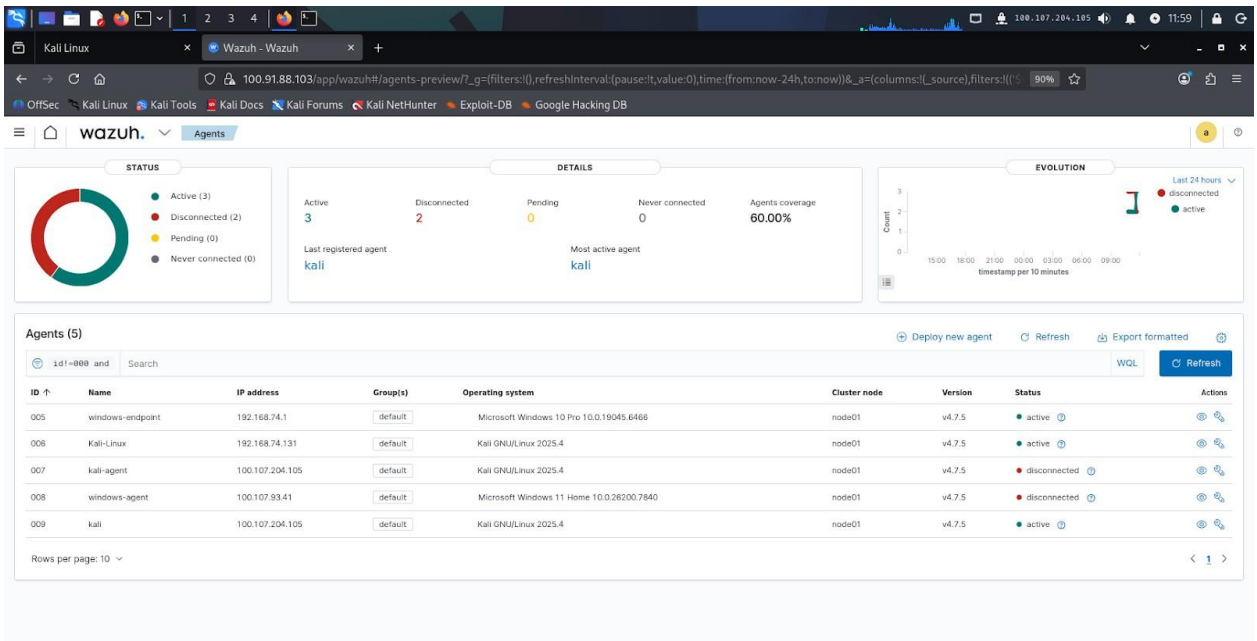
7. Gate Check Verification-

The primary requirement of Week 1 was to verify that all deployed agents were properly connected to the Wazuh Manager and actively sending monitoring data.
After deployment, the system was verified through both command-line tools and the Wazuh Dashboard.
The results confirmed that:

- All agents were reporting an "Active" status
- Agents were sending heartbeat signals to maintain communication with the manager
- Initial system and security logs were successfully transmitted
- Windows telemetry generated by Sysmon was visible in the monitoring platform

This confirmed that the centralized monitoring infrastructure was functioning correctly and that all endpoints were successfully integrated into the system.

Project review



8. Results:

The Week 1 deployment **successfully established** the foundational infrastructure for the Sentient Shield EDR Grid.

The key achievements included:

- Successful deployment of the Wazuh Manager on Ubuntu 24.04
- Deployment of Wazuh agents on Kali Linux and Windows 11
- Integration of Sysmon for enhanced Windows telemetry
- Secure network connectivity established using Tailscale
- Successful verification that all agents were active and transmitting logs
- The centralized monitoring environment is now fully operational.

Week 2 Report

1. Objective-

The objective of Week 2 was to implement **detection logic** within the SOC monitoring infrastructure developed in Week 1. This was achieved by configuring detection mechanisms within **Wazuh** to identify suspicious system activity and potential vulnerabilities.

The tasks performed during this phase included:

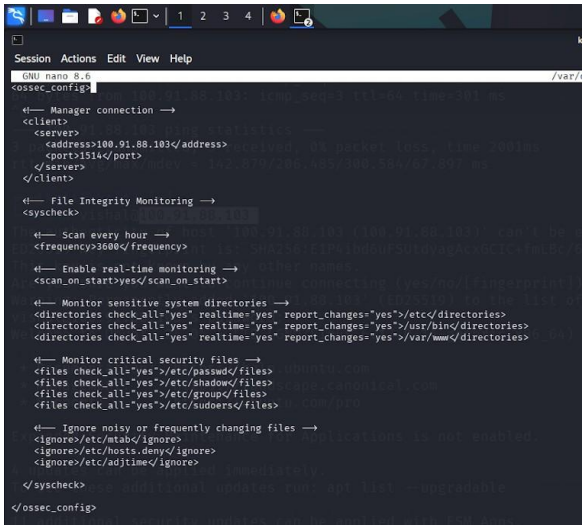
- Configuring **File Integrity Monitoring (FIM)** to monitor sensitive directories on the Kali Linux agent
- Writing **custom XML decoders** to interpret proprietary application log formats
- Creating **custom detection rules** to generate high-severity alerts
- Enabling the **Vulnerability Detector module** to identify known software vulnerabilities

- Verifying that alerts are generated when monitored files are modified

These configurations strengthened the detection capabilities of the **Sentient Shield EDR Grid** and enabled the system to identify unauthorized file modifications and suspicious activities.

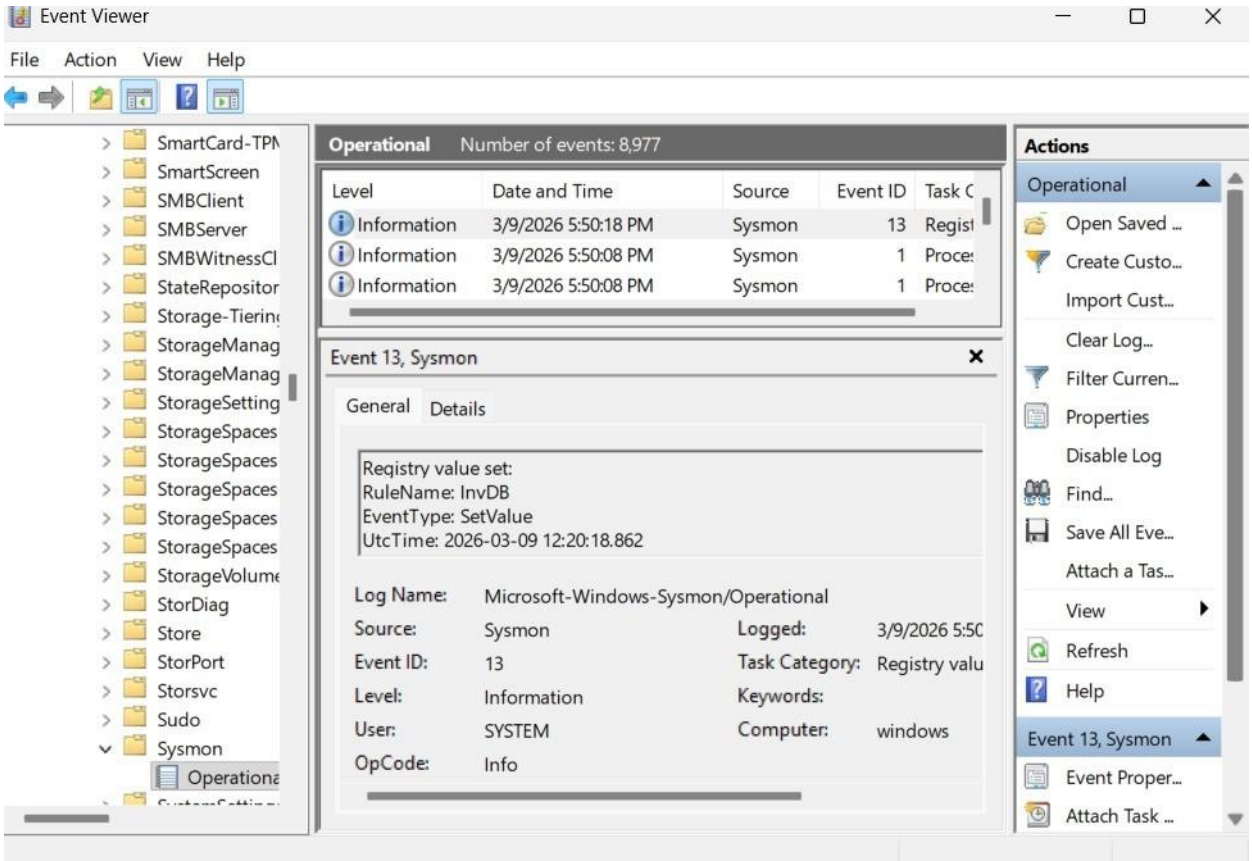
2. File Integrity Monitoring Configuration-

- File Integrity Monitoring (FIM) was configured on the Kali Linux agent to monitor sensitive directories and security-critical files.
- This feature of **Wazuh** detects unauthorized modifications to important system files and directories.The configuration was implemented in the **ossec.conf** file of the Wazuh agent.



In Windows 10 sysmon:

Project review



3. Custom XML Decoder Implementation

To analyze proprietary application logs, a custom XML decoder was created on the **Wazuh Manager.

→ sudo nano /var/ossec/etc/decoders/local_decoder.xml

The following decoder was added:

```
<decoder name="custom_app_log">
 <prematch>^\[ALERT\]</prematch>
 <regex>USER=(\w+)\sACTION=(\w+)\sFILE=(\S+)</regex>
 <order>user action file</order>
 </decoder>
```

This decoder extracts the following information from the log entry:

- Username performing the action
- Action performed by the user
- File affected by the action

This allows **Wazuh** to properly interpret and analyze logs generated by proprietary applications.

4. Custom Detection Rule Implementation-

After creating the decoder, a custom rule was implemented to trigger a **high-severity alert** when such log events are detected.

Project review

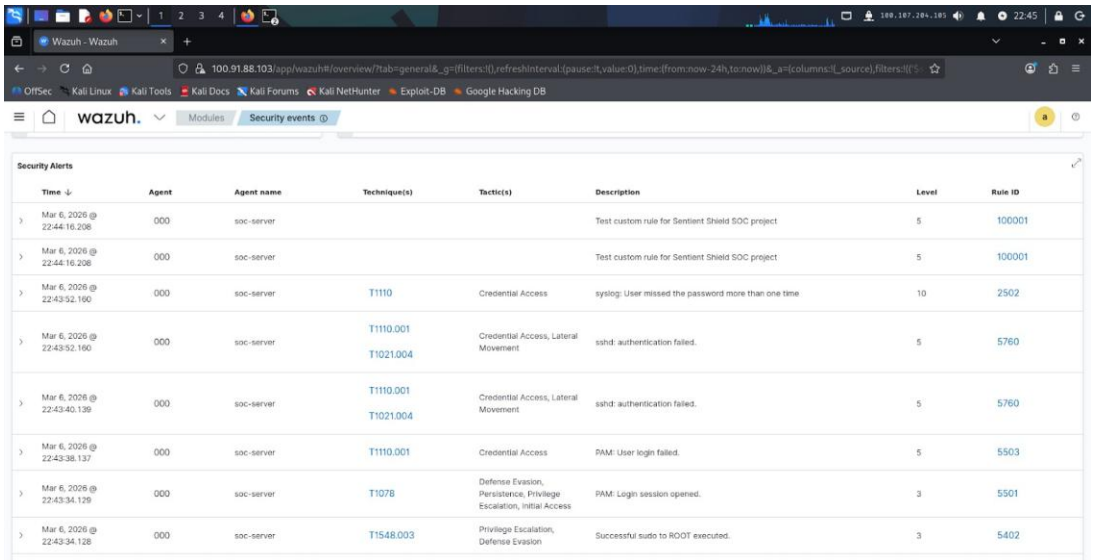
Command used:
→ sudo nano /var/ossec/etc/rules/local_rules.xml
Rule configuration:

```
<rule id="100200" level="12">
 <if_matched_sid>custom_app_log</if_matched_sid>
 <description>Critical action in proprietary application log</description>
 <mitre>T1046</mitre>
</rule>
```

Explanation:

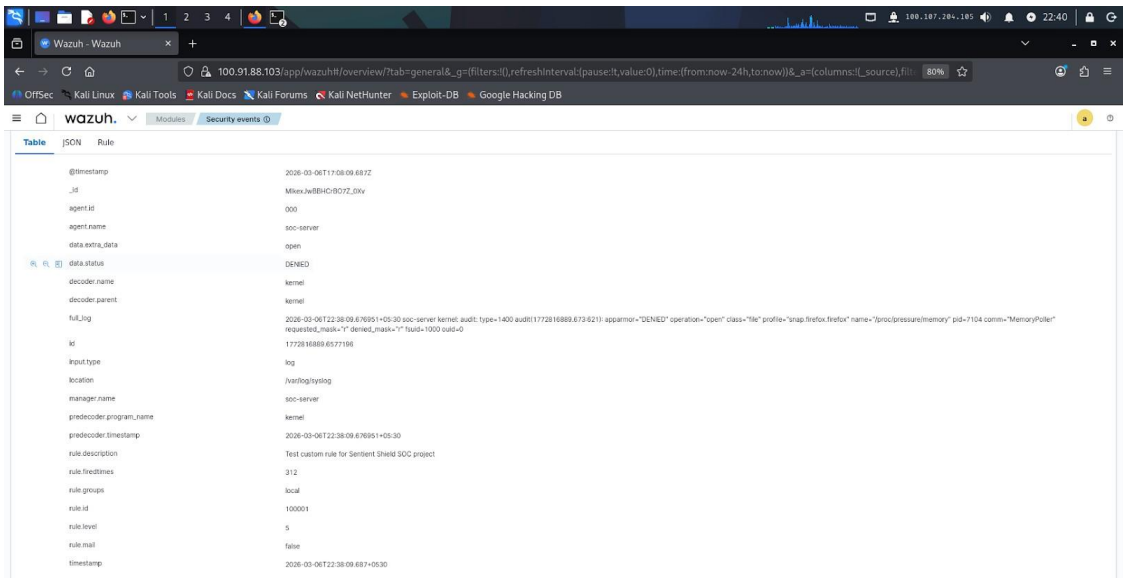
- **Rule ID:** Unique identifier for the rule
- **Severity Level:** 12 (high severity alert)
- **MITRE Mapping:** Linked to **T1046 Network Service Discovery**

After adding the rule, the manager service was restarted:
→ sudo systemctl restart wazuh-manager



Time	Agent	Agent name	Technique(s)	Tactic(s)	Description	Level	Rule ID
Mar 6, 2026 @ 22:44:16.208	000	soc-server			Test custom rule for Sentient Shield SOC project	5	100001
Mar 6, 2026 @ 22:44:16.208	000	soc-server			Test custom rule for Sentient Shield SOC project	5	100001
Mar 6, 2026 @ 22:43:52.160	000	soc-server	T1110	Credential Access	syslog: User missed the password more than one time	10	2502
Mar 6, 2026 @ 22:43:52.160	000	soc-server	T1110.001 T1021.004	Credential Access, Lateral Movement	sshd: authentication failed.	5	5760
Mar 6, 2026 @ 22:43:40.139	000	soc-server	T1110.001 T1021.004	Credential Access, Lateral Movement	sshd: authentication failed.	5	5760
Mar 6, 2026 @ 22:43:38.137	000	soc-server	T1110.001	Credential Access	PAM: User login failed.	5	5503
Mar 6, 2026 @ 22:43:34.129	000	soc-server	T1078	Defense Evasion, Persistence, Privilege Escalation, Initial Access	PAM: Login session opened.	3	5501
Mar 6, 2026 @ 22:43:34.128	000	soc-server	T1548.003	Privilege Escalation, Defense Evasion	Successful sudo to ROOT executed.	3	5402

Project review



5. Enabling the Vulnerability Detector-

The **Vulnerability Detector** module in **Wazuh** was enabled to identify known vulnerabilities in installed software packages on monitored systems.

Command used:

→ sudo nano /var/ossec/etc/ossec.conf

Configuration added:

```
<vulnerability-detector>
 <enabled>yes</enabled>
 <interval>5m</interval>
 <run_on_start>yes</run_on_start>
</vulnerability-detector>
```

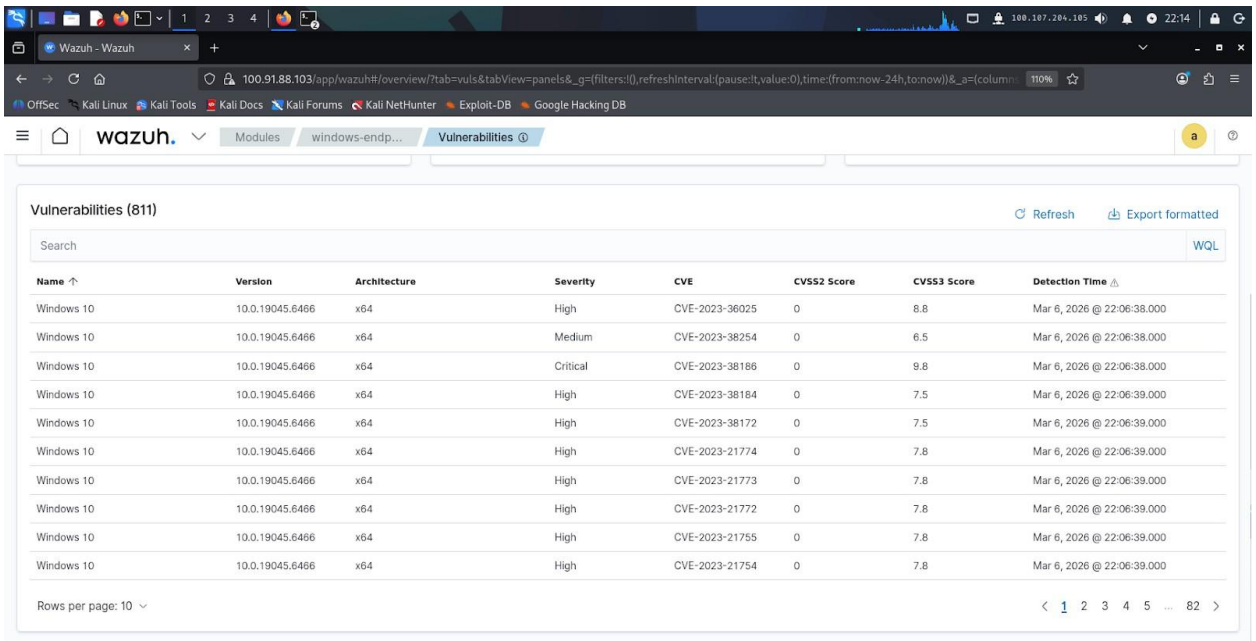
Explanation:

- **enabled:** Activates vulnerability scanning
- **interval:** Performs vulnerability checks every 5 minutes
- **run_on_start:** Executes a scan when the manager service starts

The manager service was restarted to apply the configuration:

→ sudo systemctl restart wazuh-manager

Project review



Name	Version	Architecture	Severity	CVE	CVSS2 Score	CVSS3 Score	Detection Time
Windows 10	10.0.19045.6466	x64	High	CVE-2023-36025	0	8.8	Mar 6, 2026 @ 22:06:38.000
Windows 10	10.0.19045.6466	x64	Medium	CVE-2023-38254	0	6.5	Mar 6, 2026 @ 22:06:38.000
Windows 10	10.0.19045.6466	x64	Critical	CVE-2023-38186	0	9.8	Mar 6, 2026 @ 22:06:38.000
Windows 10	10.0.19045.6466	x64	High	CVE-2023-38184	0	7.5	Mar 6, 2026 @ 22:06:39.000
Windows 10	10.0.19045.6466	x64	High	CVE-2023-38172	0	7.5	Mar 6, 2026 @ 22:06:39.000
Windows 10	10.0.19045.6466	x64	High	CVE-2023-21774	0	7.8	Mar 6, 2026 @ 22:06:39.000
Windows 10	10.0.19045.6466	x64	High	CVE-2023-21773	0	7.8	Mar 6, 2026 @ 22:06:39.000
Windows 10	10.0.19045.6466	x64	High	CVE-2023-21772	0	7.8	Mar 6, 2026 @ 22:06:39.000
Windows 10	10.0.19045.6466	x64	High	CVE-2023-21755	0	7.8	Mar 6, 2026 @ 22:06:39.000
Windows 10	10.0.19045.6466	x64	High	CVE-2023-21754	0	7.8	Mar 6, 2026 @ 22:06:39.000

6. Detection Testing-

Several tests were performed to verify that the detection system was functioning correctly.

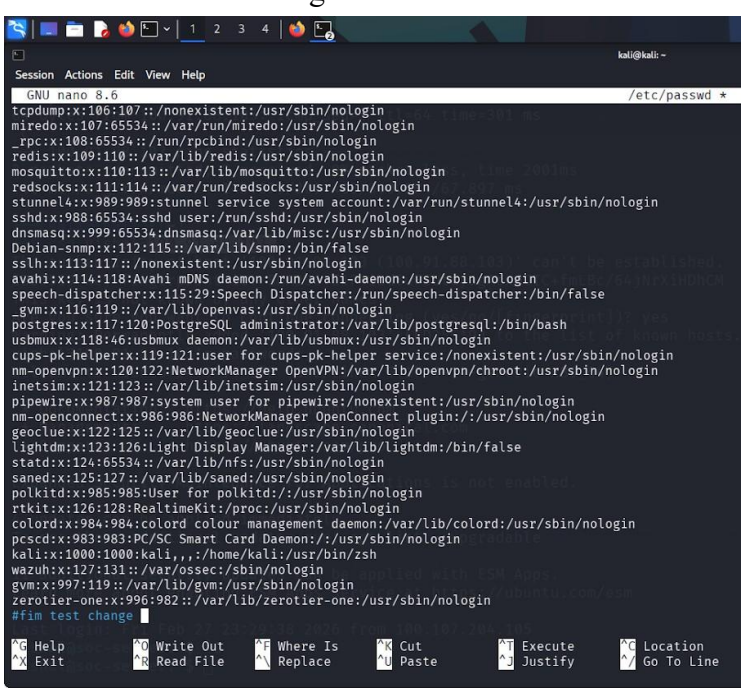
Test 1 – Modifying a Critical System File:

A modification was made to the **/etc/passwd** file to simulate unauthorized system modification.

Command used:

→ sudo nano /etc/passwd

Since this file is monitored by the File Integrity Monitoring module, the modification triggered an alert in the monitoring dashboard.



Test 2 – Simulating a Web Server Attack:
To simulate a web attack scenario, a suspicious file was created in the monitored web directory.
Command used:
→ sudo touch /var/www/malicious.php
The creation of this file generated an alert indicating that a new file had been added to a monitored directory. This type of activity may represent a **web shell upload attack**.
After saving the file, the Wazuh Manager detected the modification and generated an alert.

7. Gate Check Verification:
The Gate Check requirement for Week 2 was to verify that modifying a file in a monitored directory triggers a **high-severity alert within a short time period**.
During testing:

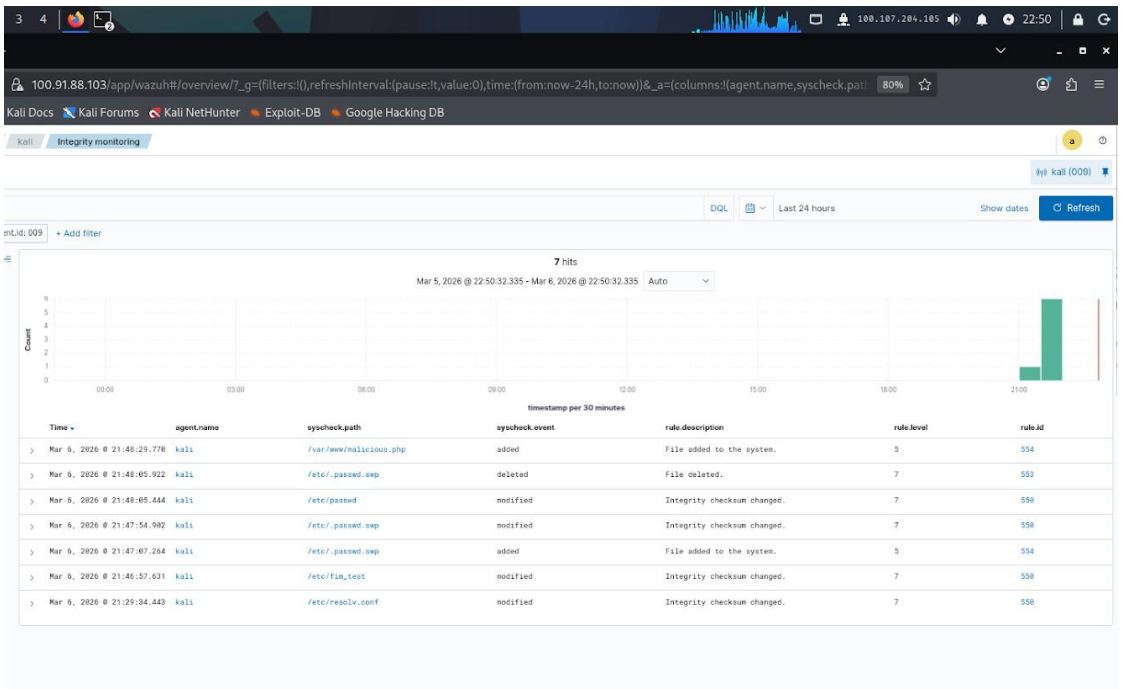
- File modifications were successfully detected.
- Alerts appeared in the Wazuh Dashboard.
- Alerts were generated within approximately **5 seconds** of the file modification.

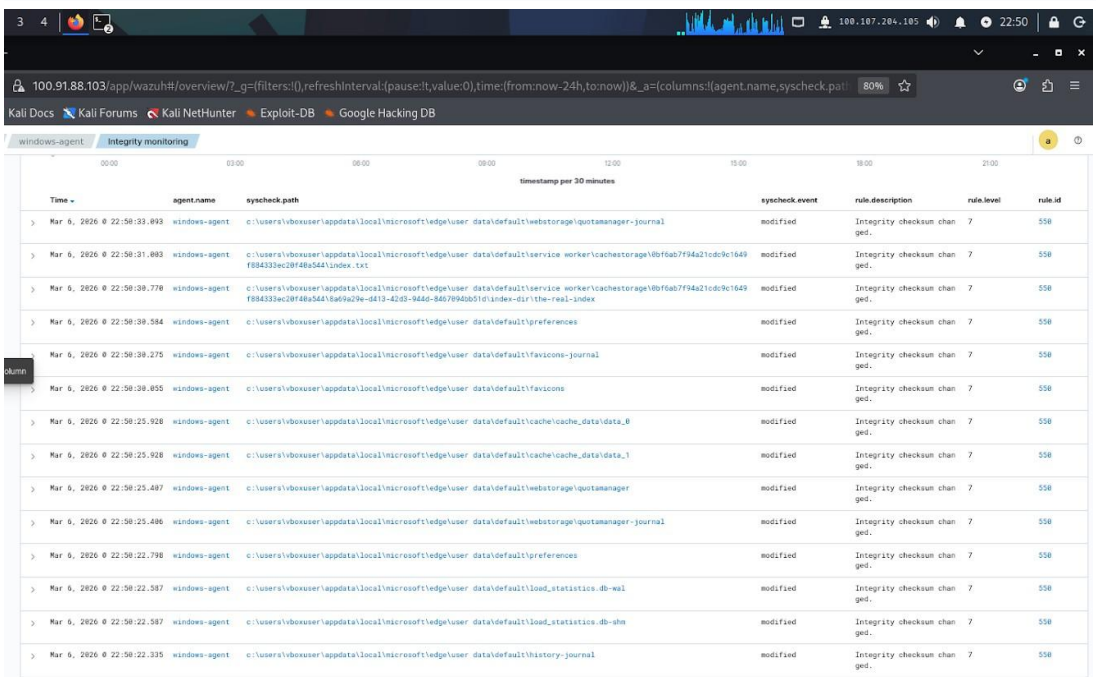
The alert contained the following information:

- Host system generating the event
- File path where the modification occurred
- Type of change detected
- Timestamp of the event
- Assigned severity level

This confirmed that the **File Integrity Monitoring and detection rules were functioning correctly**.

Project review





8. Results-

The Week 2 implementation successfully improved the detection capabilities of the **Sentient Shield EDR Grid**.

Key achievements included:

- Successful configuration of File Integrity Monitoring (FIM)
- Implementation of custom XML decoders
- Creation of high-severity detection rules

- Activation of the Vulnerability Detector module
- Successful detection of unauthorized file modifications

Week 3 Report:

1. Objective-

The objective of Week 3 was to implement an **Active Response mechanism** within the SOC monitoring infrastructure. This allows the system not only to detect malicious activity but also to **automatically respond to attacks**.

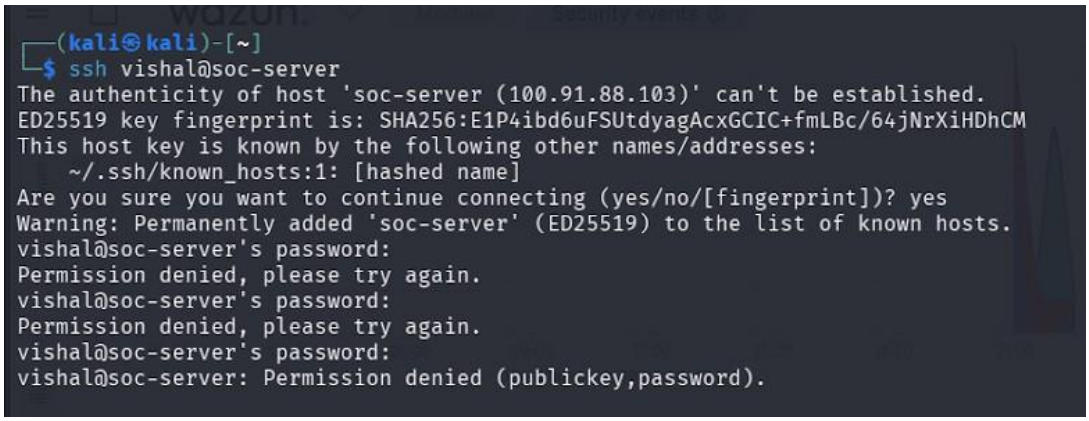
This was achieved using the Active Response capabilities of **Wazuh**, which enable automated security actions when specific detection rules are triggered.

The scenario implemented this week involved detecting and mitigating an **SSH brute force attack**. When multiple failed SSH login attempts were detected, the system automatically executed a **firewall-drop command** to block the attacker's IP address.

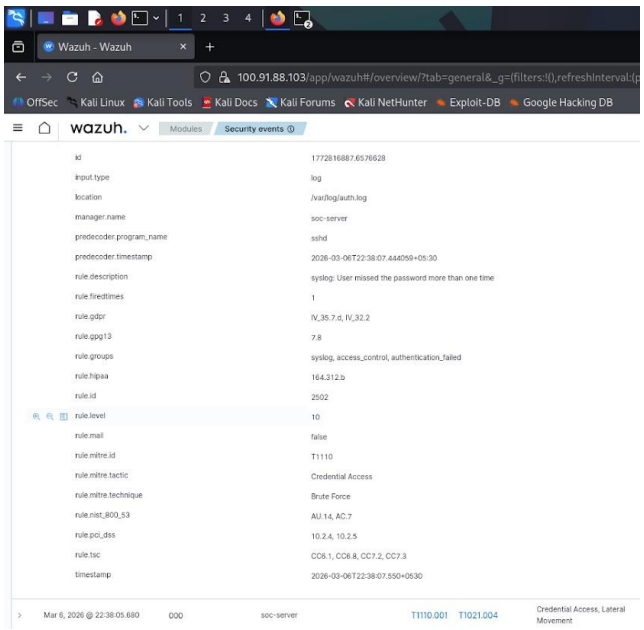
The objectives of this task included:

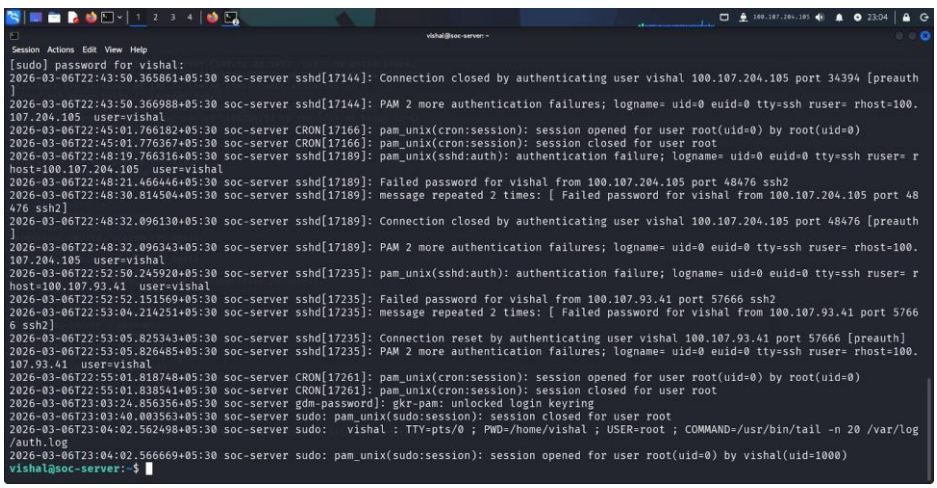
- Configuring **Active Response scripts** in the Wazuh Manager
- Defining a detection scenario for **SSH brute force attacks**
- Automatically blocking the attacker's IP address using firewall rules
- Testing the response mechanism using **Hydra**

This configuration effectively enables the system to function as an **Intrusion Prevention System (IPS)**.



Project review





2. Configuration on the Wazuh Manager-

The Active Response mechanism was configured on the **Wazuh Manager** to automatically block suspicious IP addresses when a brute force attack is detected.

The firewall instruction was added in the Wazuh configuration file so that when a specific rule related to SSH brute force attempts is triggered, the **firewall-drop command** is executed.

Example configuration:

```
<active-response>
 <command>firewall-drop</command>
 <location>local</location>
 <rules_id>5710</rules_id>
</active-response>
```

Explanation:

- **command:** Specifies the action script to run

- **location:** Indicates that the command will run on the local agent
- **rules_id:** Refers to the rule responsible for detecting SSH brute force attempts

Once configured, the Wazuh Manager is capable of automatically instructing the host system to block malicious IP addresses through the firewall.

3. Installation of Hydra on the Kali Agent-

To simulate a brute force attack, the penetration testing tool **Hydra** was installed on the Kali Linux agent machine.

Kali Linux already includes Hydra in its default toolset. If not present, it can be installed using:

→ sudo apt install hydra

This tool was used to generate multiple login attempts against the SSH service running on the monitored host.

4. Brute Force Attack Simulation-

The SSH brute force attack was simulated using Hydra from the Kali Linux machine targeting the monitored host.

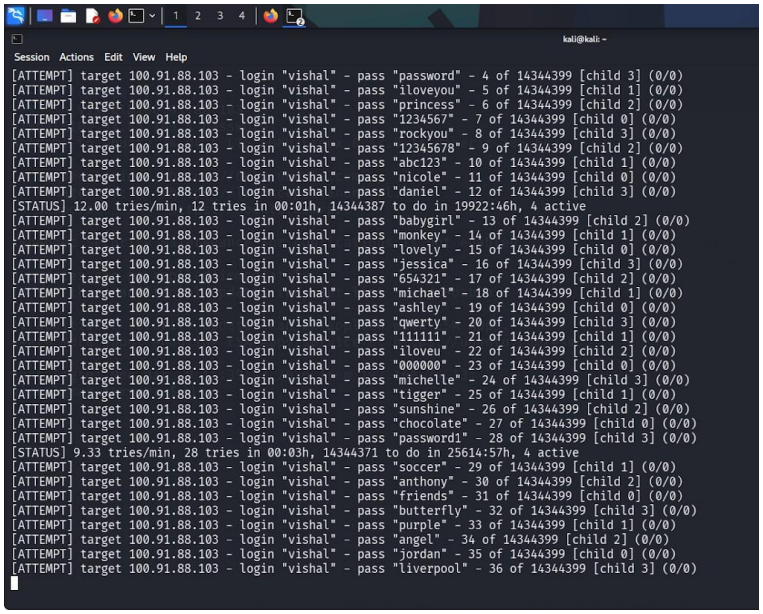
The following command was executed:

→ hydra -l vishal -P /usr/share/wordlists/rockyou.txt ssh://100.91.88.103 -t 4 -V

Command Explanation

- **-l vishal** → Specifies the username being targeted
- **-P /usr/share/wordlists/rockyou.txt** → Uses the *rockyou* password wordlist
- **ssh://100.91.88.103** → Target SSH service and host IP address
- **-t 4** → Performs four parallel login attempts
- **-V** → Displays verbose output showing each attempt

This command attempts multiple password combinations from the wordlist in order to gain access to the target system.



```
kali@kali: ~
Session  Actions  Edit  View  Help
[ATTEMPT] target 100.91.88.103 - login "vishal" - pass "password" - 4 of 14344399 [child 3] (0/0)
[ATTEMPT] target 100.91.88.103 - login "vishal" - pass "iloveyou" - 5 of 14344399 [child 1] (0/0)
[ATTEMPT] target 100.91.88.103 - login "vishal" - pass "princess" - 6 of 14344399 [child 2] (0/0)
[ATTEMPT] target 100.91.88.103 - login "vishal" - pass "1234567" - 7 of 14344399 [child 0] (0/0)
[ATTEMPT] target 100.91.88.103 - login "vishal" - pass "rockyou" - 8 of 14344399 [child 3] (0/0)
[ATTEMPT] target 100.91.88.103 - login "vishal" - pass "12345678" - 9 of 14344399 [child 2] (0/0)
[ATTEMPT] target 100.91.88.103 - login "vishal" - pass "abc123" - 10 of 14344399 [child 1] (0/0)
[ATTEMPT] target 100.91.88.103 - login "vishal" - pass "nicole" - 11 of 14344399 [child 0] (0/0)
[ATTEMPT] target 100.91.88.103 - login "vishal" - pass "daniel" - 12 of 14344399 [child 3] (0/0)
[STATUS] 12.00 tries/min, 12 tries in 00:01h, 14344387 to do in 19922:46h, 4 active
[ATTEMPT] target 100.91.88.103 - login "vishal" - pass "babygirl" - 13 of 14344399 [child 2] (0/0)
[ATTEMPT] target 100.91.88.103 - login "vishal" - pass "monkey" - 14 of 14344399 [child 1] (0/0)
[ATTEMPT] target 100.91.88.103 - login "vishal" - pass "lovely" - 15 of 14344399 [child 0] (0/0)
[ATTEMPT] target 100.91.88.103 - login "vishal" - pass "jessica" - 16 of 14344399 [child 3] (0/0)
[ATTEMPT] target 100.91.88.103 - login "vishal" - pass "654321" - 17 of 14344399 [child 2] (0/0)
[ATTEMPT] target 100.91.88.103 - login "vishal" - pass "michael" - 18 of 14344399 [child 1] (0/0)
[ATTEMPT] target 100.91.88.103 - login "vishal" - pass "ashley" - 19 of 14344399 [child 0] (0/0)
[ATTEMPT] target 100.91.88.103 - login "vishal" - pass "qwerty" - 20 of 14344399 [child 3] (0/0)
[ATTEMPT] target 100.91.88.103 - login "vishal" - pass "111111" - 21 of 14344399 [child 1] (0/0)
[ATTEMPT] target 100.91.88.103 - login "vishal" - pass "iloveu" - 22 of 14344399 [child 2] (0/0)
[ATTEMPT] target 100.91.88.103 - login "vishal" - pass "000000" - 23 of 14344399 [child 0] (0/0)
[ATTEMPT] target 100.91.88.103 - login "vishal" - pass "michelle" - 24 of 14344399 [child 3] (0/0)
[ATTEMPT] target 100.91.88.103 - login "vishal" - pass "tigger" - 25 of 14344399 [child 1] (0/0)
[ATTEMPT] target 100.91.88.103 - login "vishal" - pass "sunshine" - 26 of 14344399 [child 2] (0/0)
[ATTEMPT] target 100.91.88.103 - login "vishal" - pass "chocolate" - 27 of 14344399 [child 0] (0/0)
[ATTEMPT] target 100.91.88.103 - login "vishal" - pass "password1" - 28 of 14344399 [child 3] (0/0)
[STATUS] 9.33 tries/min, 28 tries in 00:03h, 14344371 to do in 25614:57h, 4 active
[ATTEMPT] target 100.91.88.103 - login "vishal" - pass "soccer" - 29 of 14344399 [child 1] (0/0)
[ATTEMPT] target 100.91.88.103 - login "vishal" - pass "anthony" - 30 of 14344399 [child 2] (0/0)
[ATTEMPT] target 100.91.88.103 - login "vishal" - pass "friends" - 31 of 14344399 [child 0] (0/0)
[ATTEMPT] target 100.91.88.103 - login "vishal" - pass "butterfly" - 32 of 14344399 [child 3] (0/0)
[ATTEMPT] target 100.91.88.103 - login "vishal" - pass "purple" - 33 of 14344399 [child 1] (0/0)
[ATTEMPT] target 100.91.88.103 - login "vishal" - pass "angel" - 34 of 14344399 [child 2] (0/0)
[ATTEMPT] target 100.91.88.103 - login "vishal" - pass "jordan" - 35 of 14344399 [child 0] (0/0)
[ATTEMPT] target 100.91.88.103 - login "vishal" - pass "liverpool" - 36 of 14344399 [child 3] (0/0)
```

5. Detection and Automatic Blocking-

During the brute force attack, the repeated failed login attempts were detected by **Wazuh**.
Once the system identified the attack pattern, the following actions occurred automatically:

1. The Wazuh agent detected multiple failed SSH login attempts.
2. The detection rule associated with brute force activity was triggered.
3. The Active Response mechanism executed the **firewall-drop command**.
4. The attacker's IP address was automatically added to the host firewall block list.

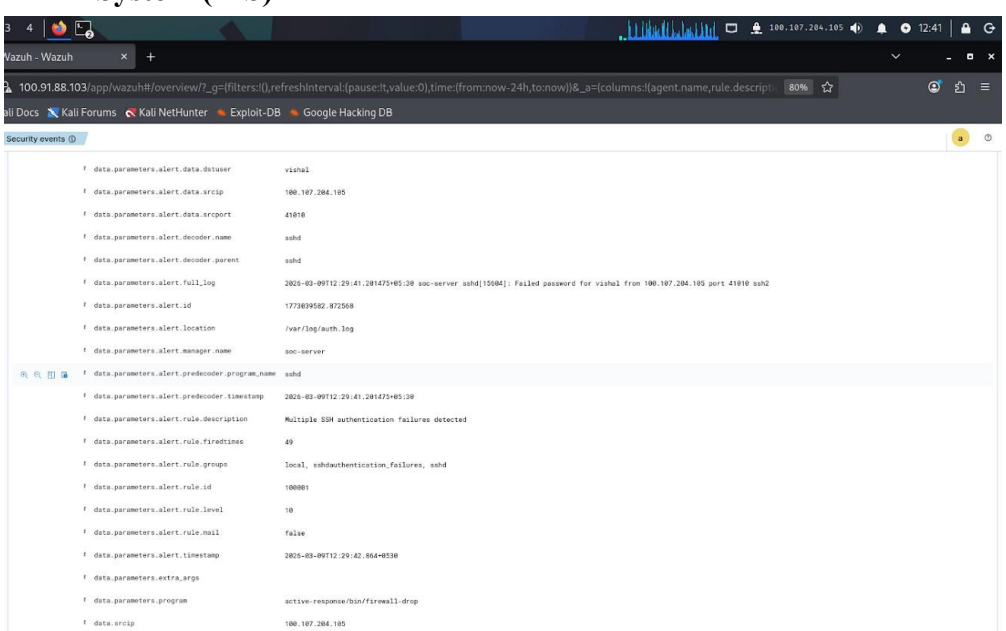
As a result, the attacking machine was blocked from continuing further login attempts.
The blocking action was visible in:

- **Wazuh logs**, showing that the host executed the firewall-drop command
- **Wazuh Dashboard**, where an alert indicated that the attacker's IP address had been successfully blocked

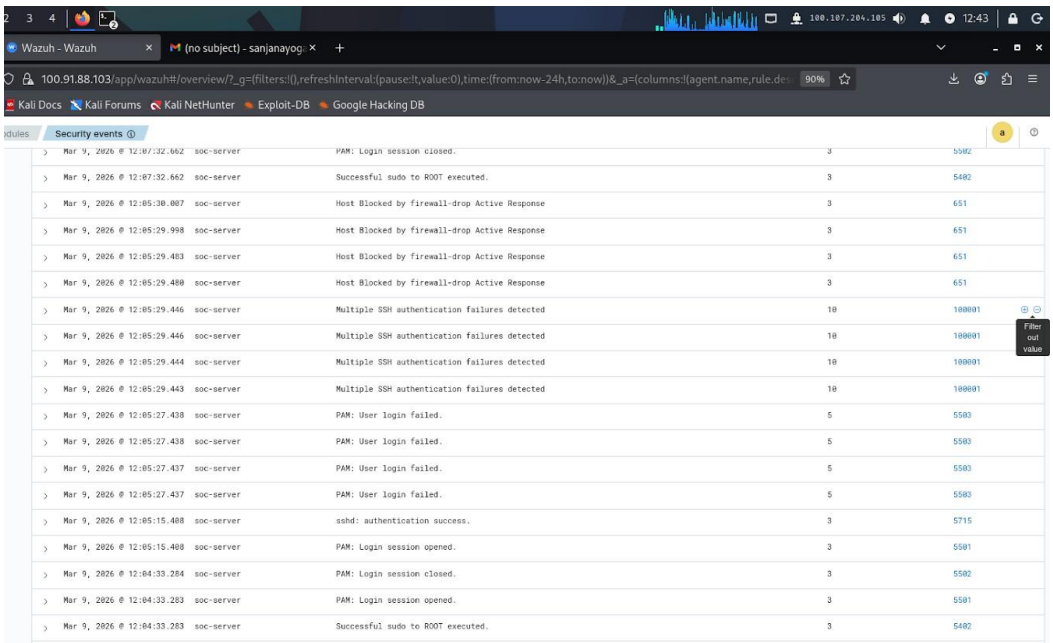
This confirmed that the automated response mechanism was functioning correctly.

6. Gate Check Verification-

- The Gate Check requirement for Week 3 was to confirm that the attacker's IP address would be automatically blocked during a brute force attack.
- This was verified through the Hydra attack simulation. When the attack was executed, the system detected the repeated login failures and triggered the Active Response mechanism.
- The attacker's IP address was automatically blocked by the host firewall, preventing further SSH attempts. The blocking event was recorded both in the **system logs** and the **Wazuh Dashboard**, confirming successful implementation of the **Intrusion Prevention System (IPS)**.



Project review



7. Conclusion-

- Week 3 focused on implementing automated defensive actions within the SOC monitoring infrastructure.
- By configuring the **Active Response mechanism in Wazuh**, the **Sentient Shield EDR Grid** can now automatically detect and block brute force attacks targeting the SSH service.
- This capability significantly improves the security posture of the system by reducing response time and preventing attackers from continuing malicious activities.
- The successful implementation of this feature demonstrates the transition from a **passive monitoring system to an active intrusion prevention platform**.

Week 4:

1. Objective

The objective of Week 4 was to simulate real-world attacker behavior in order to validate the effectiveness of the SOC monitoring infrastructure. This was done by executing controlled attack simulations that mimic common ransomware techniques.

The simulation was performed using the adversary emulation framework **Atomic Red Team**, which contains predefined tests based on the **MITRE ATT&CK** framework.

- Installing and configuring Atomic Red Team on a Windows endpoint
- Simulating ransomware-like behavior (Shadow Copy deletion)
- Monitoring the activity using **Wazuh**

2. Installation of Atomic Red Team on Windows 10 VM

Atomic Red Team was installed on the Windows 10 VM, which had the **Wazuh Agent**.

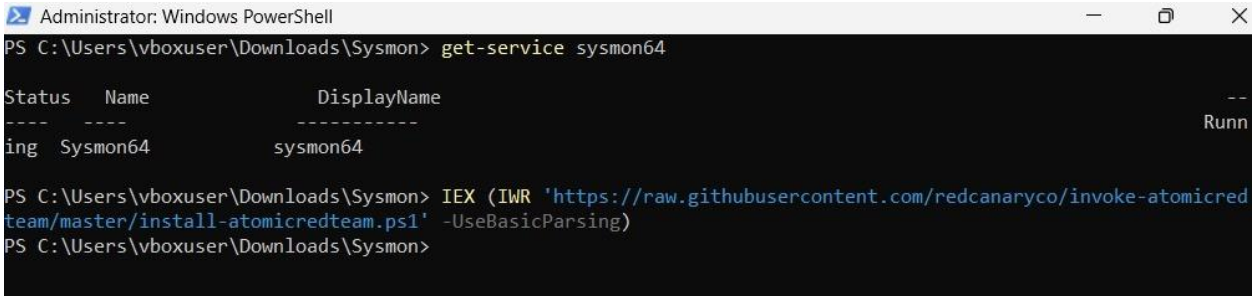
PowerShell command used:

IEX (IWR 'https://raw.githubusercontent.com/redcanaryco/invoke-atomicredteam/master/install-atomicredteam.ps1' -UseBasicParsing)

Explanation:

- **IEX** → Executes the command directly in PowerShell
- **IWR** → Downloads the installation script from GitHub
- **UseBasicParsing** → Ensures compatibility on Windows systems without IE

After execution, Atomic Red Team was ready to run individual MITRE ATT&CK technique simulations.



3. Ransomware Technique Simulation (T1490)

The ransomware simulation targeted **T1490 – Inhibit System Recovery** to delete Shadow Volume Copies.

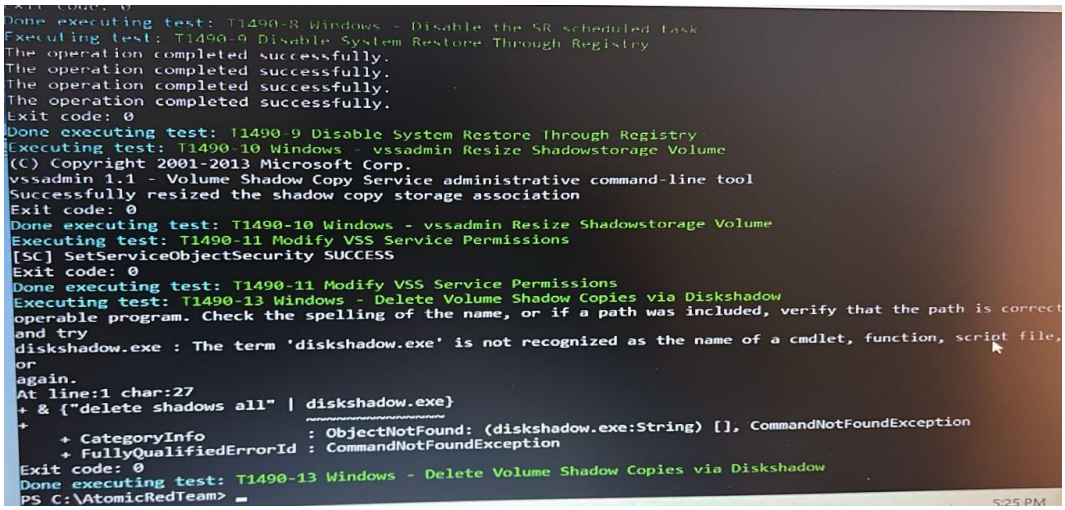
Execution command used:

Invoke-AtomicTest T1490 -Path C:\AtomicRedTeam\atomics

Explanation:

- **Invoke-AtomicTest T1490** → Runs the test for T1490 (inhibit system recovery)
- **-Path C:\AtomicRedTeam\atomics** → Specifies the location of the Atomic Red Team test scripts

This simulates ransomware behavior prior to encryption, specifically targeting system recovery mechanisms.



T1059.001 test was also performed

The T1059.001 test simulates execution of commands via PowerShell to mimic scripting-based attacks:

Invoke-AtomicTest T1059.001 -Path C:\AtomicRedTeam\atomics

- **Invoke-AtomicTest T1059.001** → Executes PowerShell scripting attack
- **-Path C:\AtomicRedTeam\atomics** → Specifies the location of test scripts

Project review

This simulates malicious command execution or lateral movement often seen in real-world attacks.

```
PS C:\AtomicRedTeam> $PathToAtomicsFolder="C:\AtomicRedTeam\atomics"
PS C:\AtomicRedTeam> Invoke-AtomicTest T1059 -Path $PathToAtomicsFolder
PathToAtomicsFolder = C:\AtomicRedTeam\atomics

Executing test: T1059-1 AutoIt Script Execution
Start-Process : This command cannot be run due to the error: The system cannot find the file specified.
    + CategoryInfo          : InvalidOperation: (:) [Start-Process], InvalidOperationException
    + FullyQualifiedErrorId : InvalidOperationException,Microsoft.PowerShell.Commands.StartProcessCommand

+     ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
+ & {Start-Process -FilePath "C:\Program Files (x86)\AutoIt3\AutoIt3.ex ...
At line:1 char:4
Exit code: 0
Done executing test: T1059-1 AutoIt Script Execution
PS C:\AtomicRedTeam>
```

4. Detection by Wazuh

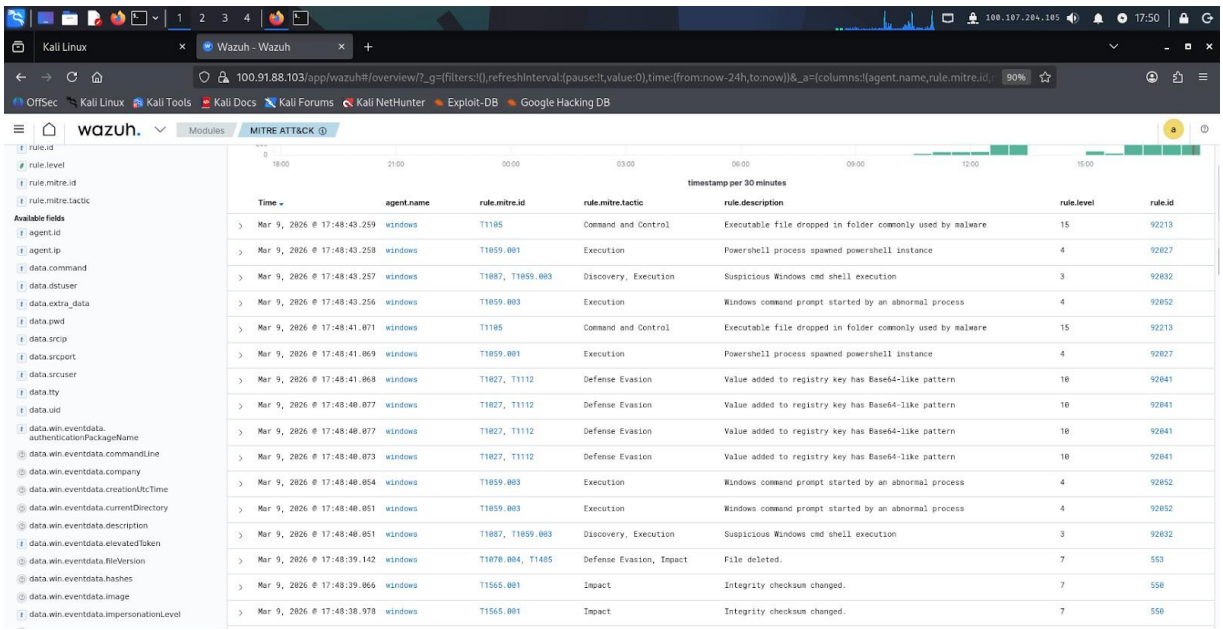
During execution of both T1490 and T1059.001 tests:

1. The Windows 10 VM generated process and security logs.
2. The **Wazuh Agent** collected and forwarded logs to the manager.
3. Detection rules for **system recovery tampering (T1490)** and **suspicious PowerShell activity (T1059.001)** triggered alerts.
4. Alerts appeared in the SOC dashboard.

Both techniques were successfully detected, confirming effective monitoring coverage.

Security Alerts

Time	Agent	Agent name	Technique(s)	Tactic(s)	Description	Level	Rule ID
Mar 9, 2026 @ 17:38:50.868	010	windows	T1078	Defense Evasion, Persistence, Privilege Escalation, Initial Access	Windows logon success.	3	60106
Mar 9, 2026 @ 17:38:49.516	010	windows	T1059.001	Execution	C:\\Windows\\SysWOW64\\SecEdit.exe binary in a suspicious location launched by C:\\Windows\\SysWOW64\\WindowsPowerShell\\v1.0\\powershell.exe	4	92066
Mar 9, 2026 @ 17:38:47.557	010	windows	T1105	Command and Control	Powershell process created an executable file in Windows root folder	9	92205
Mar 9, 2026 @ 17:38:47.471	010	windows	T1087	Discovery	Discovery activity executed	3	92031
Mar 9, 2026 @ 17:38:47.467	010	windows	T1087	Discovery	Discovery activity executed	3	92031
Mar 9, 2026 @ 17:38:47.440	010	windows	T1087	Discovery	A net.exe account discovery command was initiated	3	92039
Mar 9, 2026 @ 17:38:47.439	010	windows	T1087	Discovery	A net.exe account discovery command was initiated	3	92039
Mar 9, 2026 @ 17:38:47.424	010	windows	T1087	Discovery	A net.exe account discovery command was initiated	3	92039
Mar 9, 2026 @ 17:38:46.380	010	windows	T1087	Discovery	A net.exe account discovery command was initiated	3	92039
Mar 9, 2026 @ 17:38:46.360	010	windows	T1087	Discovery	A net.exe account discovery command was initiated	3	92039

Rows per page: 10 < 1 2 3 4 5 ... 908 >

Project review